

팍리스 실무형 일경험 프로그램

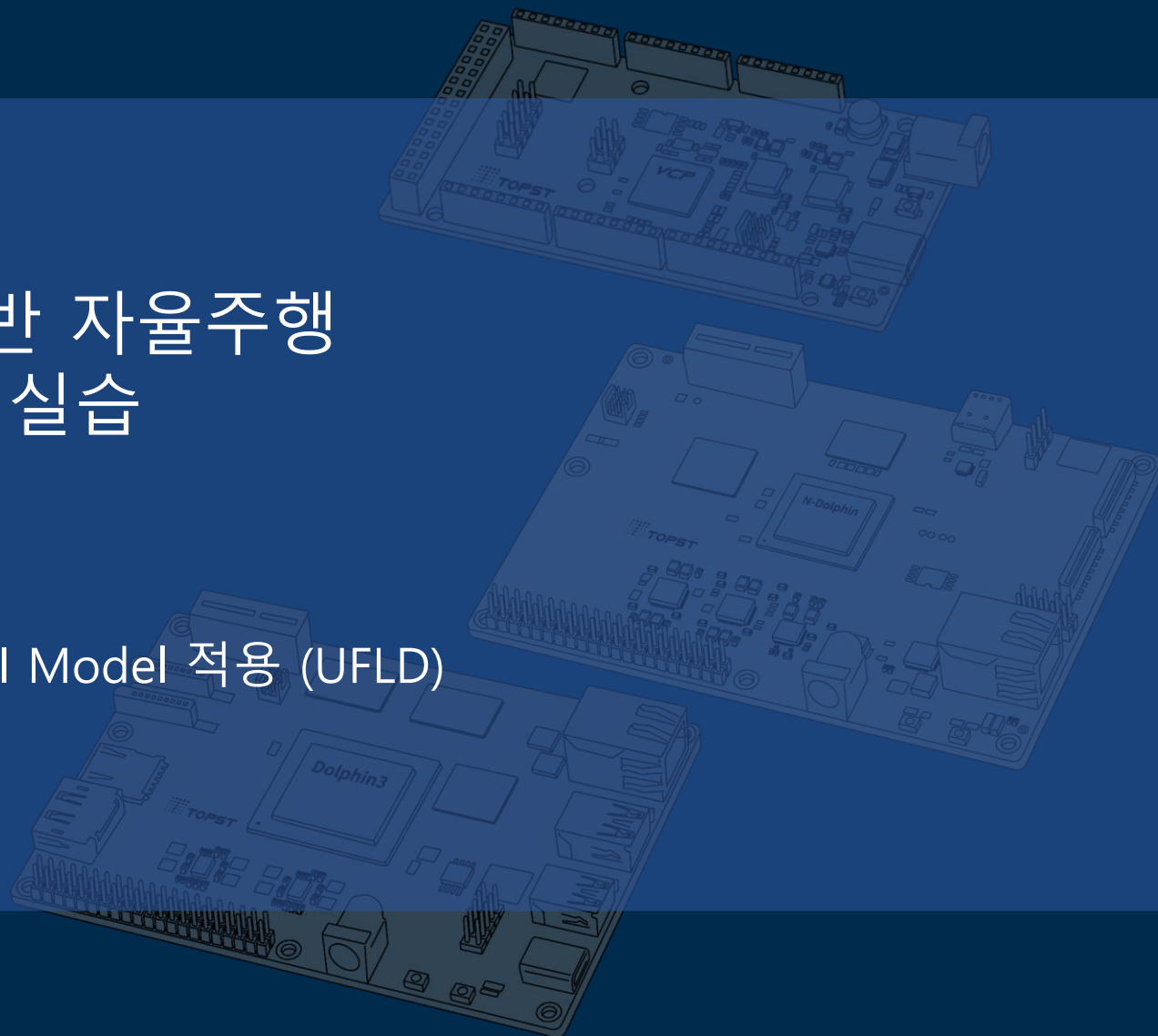
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



7일차

01 Sensing Zone – AI Model 적용 (UFLD)

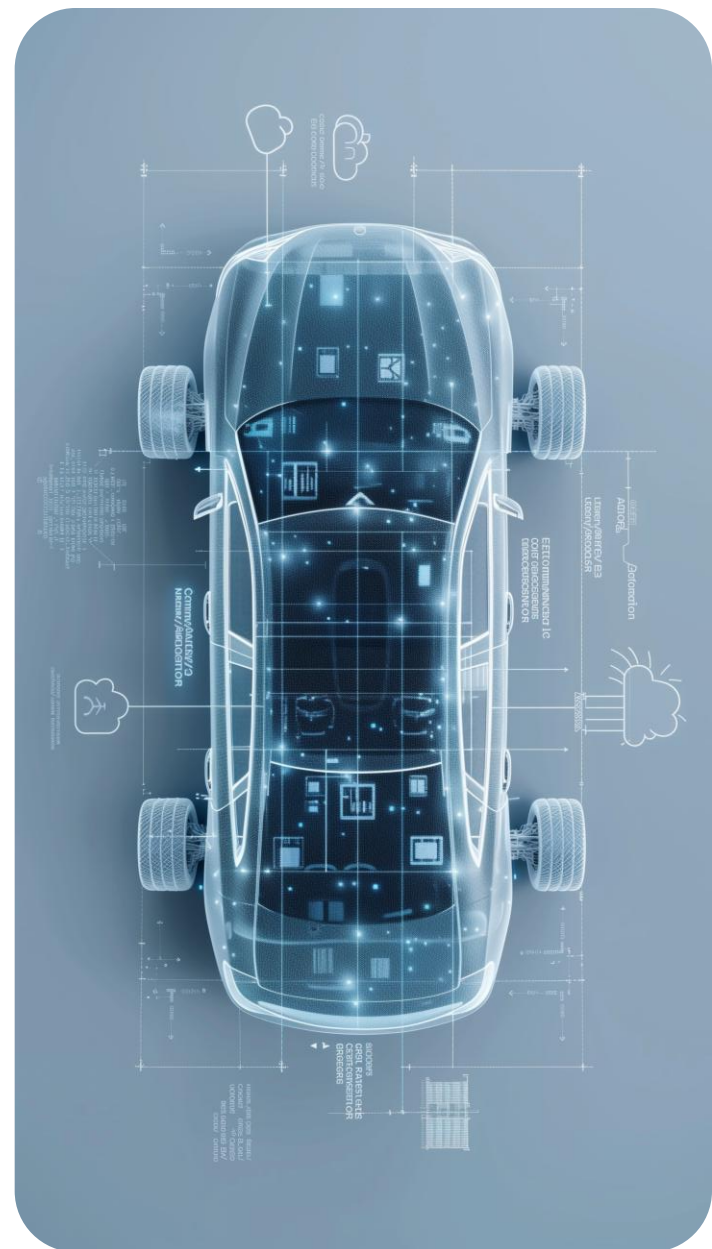
Telechips TOPST



목차

Table of Contents

모델 소개 (UFLD)	01
UFLD 모델 변환	02
UFLD 모델 후처리 과정	03

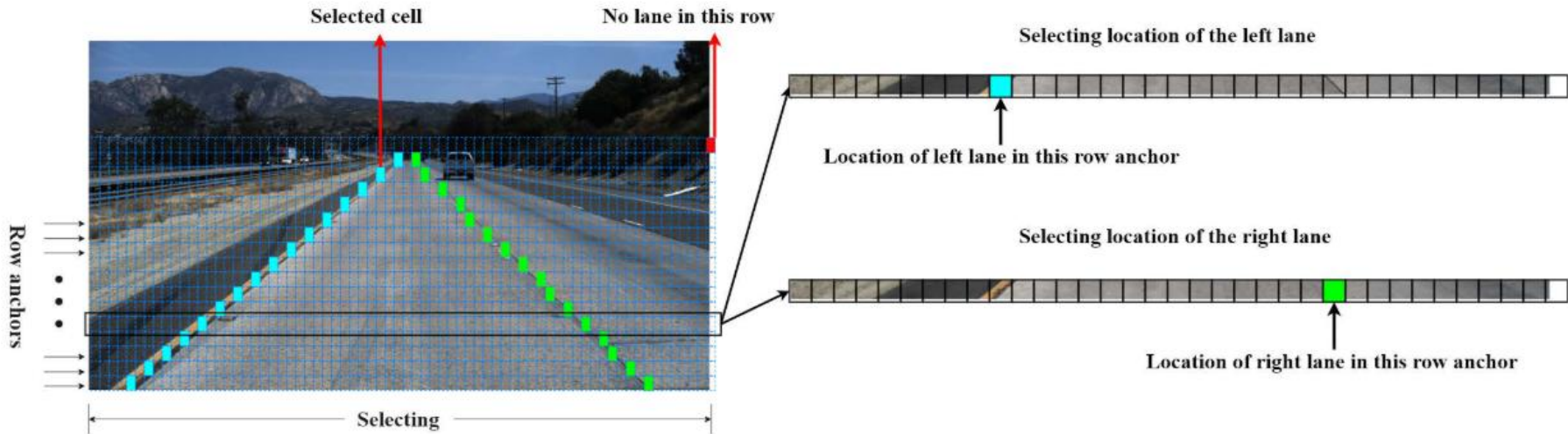


모델 소개 - UFLD (Ultra Lane Fast Detection)



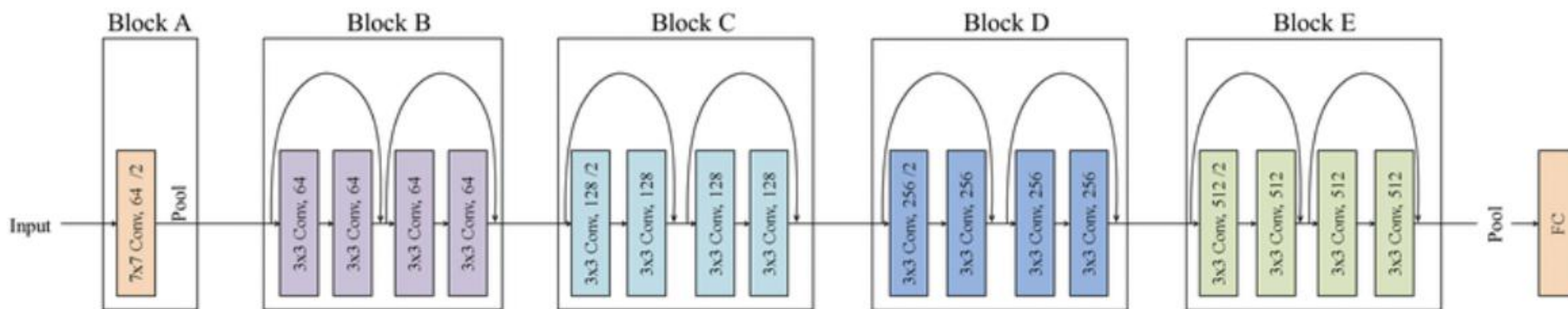
- 차선을 초고속으로 인식하기 위해 설계된 딥러닝 기반 모델
- 실시간 차선 검출에 주로 사용됨
- LaneNet과 SCNN과 같은 차선 검출 모델에 비해 가볍으면서 실시간 추론 가능
- End-to-End 방식
- 그리드 분류 기반 (Classification + 회귀) 접근

모델 소개 - UFLD (Ultra Lane Fast Detection)



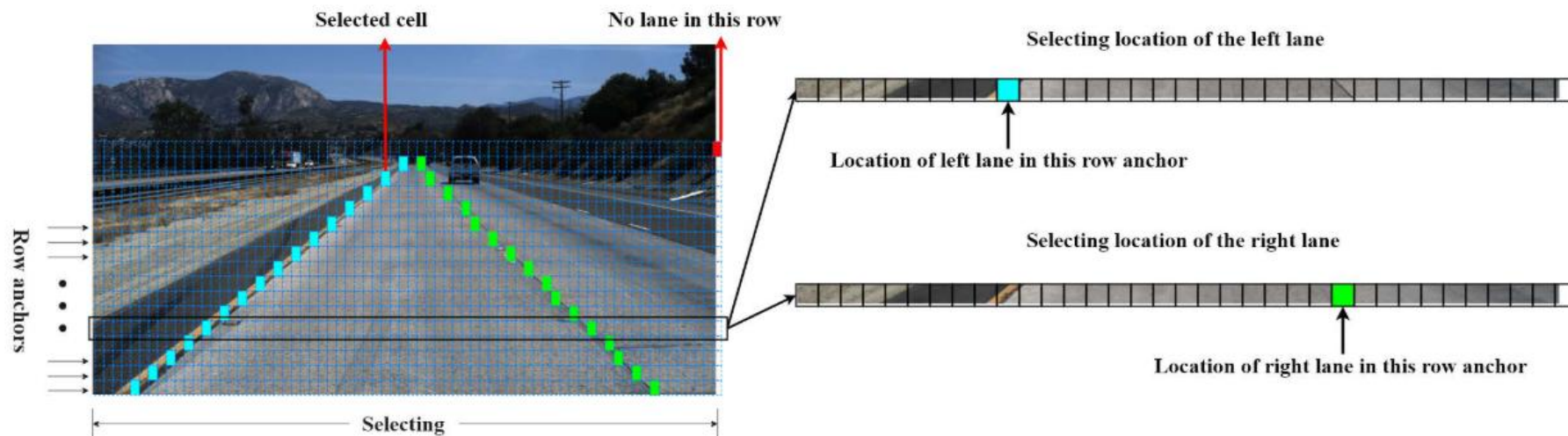
- Backbone:
 - ResNet-18
- Head:
 - 이미지를 수평 구간으로 나눠 각 row마다 lane 위치 분류
- Output:
 - 각 lane에 대해 (row 수 x grid 수) 의 classification 결과

모델 소개 - UFLD (Ultra Lane Fast Detection)



- ResNet 18:
 - 역할 : 입력 영상을 feature map으로 변환 (패턴 추출)
 - 구조 : ResNet-18의 기본 블록 (Conv + BN + ReLU + Skip) 사용
 - 특징 : 깊이가 얇아 추론 속도 빠름
 - FCN 기반이 아닌 CNN Feature Extractor 구조로 NPU 친화적

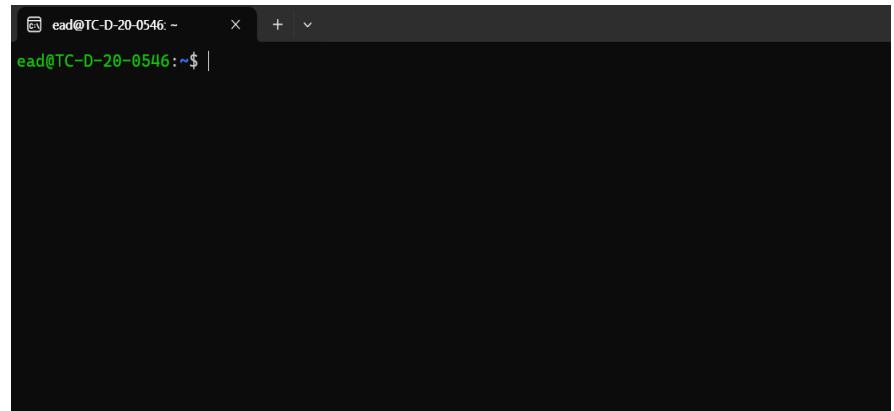
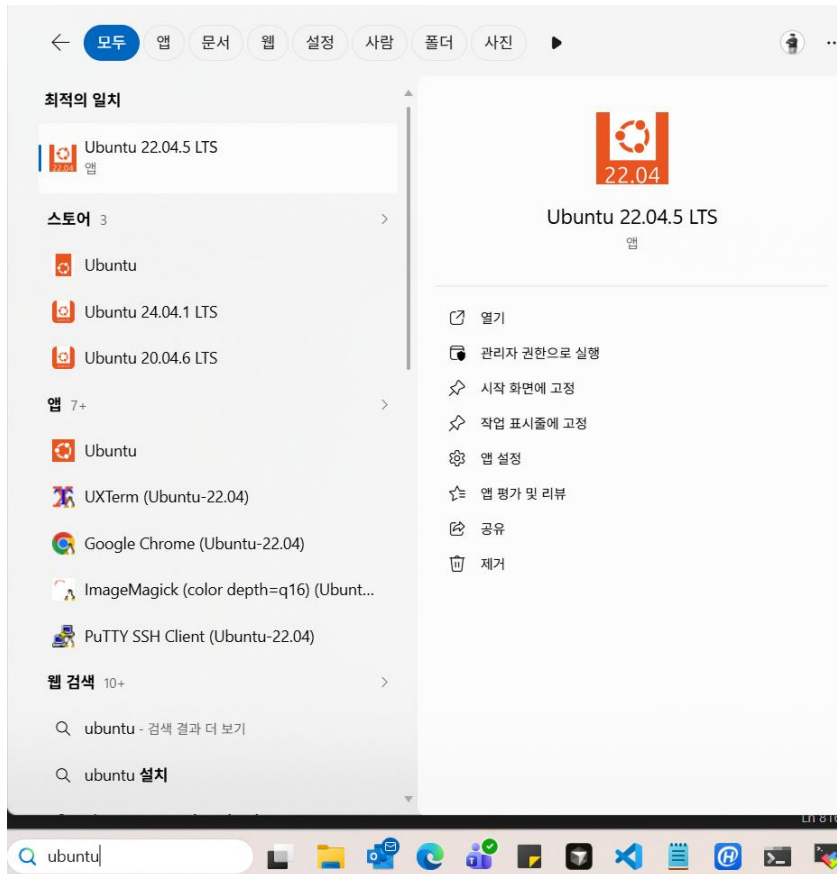
모델 소개 - UFLD (Ultra Lane Fast Detection)



- Head
 - 역할 : 추출한 Feature Map를 각 row 별 lane 위치 확률을 매핑
 - 흐름 :
 - (1) 입력 이미지를 세로 방향으로 일정 간격 만큼 분할
 - (2) Feature Map에서 각 Row Anchor에 해당하는 위치의 feature 추출
 - (3) FC 레이어를 통과하여 각 grid에 대해 softmax 확률 예측
 - (4) 차선 좌표를 보정하기 위해 가중 평균 합 계산

UFLD 모델 변환

- WSL Ubuntu 22.04 실행



UFLD 모델 변환

- tc-nn-toolkit 가상 환경 세팅

```
$ cd
```

```
$ cd ~/Work/tc-nn-toolkit
```

```
$ . venv/bin/activate
```

- Input 디렉터리에 UFLD.onnx 모델 존재하는지 확인

```
$ ls input/UFLD.onnx
```

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ ls input/UFLD.onnx  
input/UFLD.onnx
```

UFLD 모델 변환

- 모델 컨버팅 수행

```
$ python3 EnlightSDK/converter.py input/UFLD.onnx --type unknown --dataset Custom --dataset-root my_dataset_path --output output_networks/UFLD.enlight --enable-track
```

Checking arguments... done

Checking compatibility with ENLIGHT NPU

SUMMARY	
Number of compatible layers	35
Number of incompatible layers	0
Total number of layers	35

Checking compatibility with ENLIGHT NPU ... Done

Writing compatibility result to ... /home/ead/tc-nn-toolkit/tc-nn-toolkit/log/compatibility_results/UFLD_compatibility.log

Serializing Graph done.

Writing to File..... Done

Writing File path : output_networks/UFLD.enlight

Converter. done.

UFLD 모델 변환

- 모델 컨버팅 수행

```
$ python3 EnlightSDK/converter.py input/UFLD.onnx --type unknown --dataset Custom --dataset-root my_dataset_path --output output_networks/UFLD.enlight --enable-track
```

- 옵션 설명

- type : 후처리 타입 설정 (obj나 cls가 아니기에 unknown으로 설정)
- dataset : 양자화 스케일 값 추출을 위한 데이터셋
- output : 변환된 모델 output
- enable-track : 양자화 스케일 수집을 위한 옵션 활성화

UFLD 모델 변환

- 모델 양자화 수행

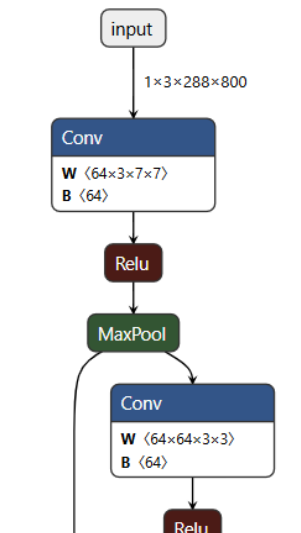
```
$ python3 EnlightSDK/quantizer.py output_networks/UFLD.enlight --output  
output_networks/UFLD_quantized.enlight
```

```
optimizing ..... done.  
quantization..... done.  
End sanity check for custom quantization  
Start MakeCompilerFriendly optimizer  
Traceback (most recent call last):  
  File "EnlightSDK/quantizer.py", line 265, in <module>  
    main()  
  File "EnlightSDK/quantizer.py", line 255, in main  
    model = prepare_model(args)  
  File "EnlightSDK/quantizer.py", line 100, in prepare_model  
    net = optimizer.optimize(net)  
  File "<frozen optimizer>", line 41, in optimize  
  File "<frozen optimizer>", line 98, in apply  
  File "<frozen optimizations.make_compiler_friendly>", line 56, in apply  
  File "<frozen optimizations.make_compiler_friendly>", line 80, in handle_reshape  
NotImplementedError
```

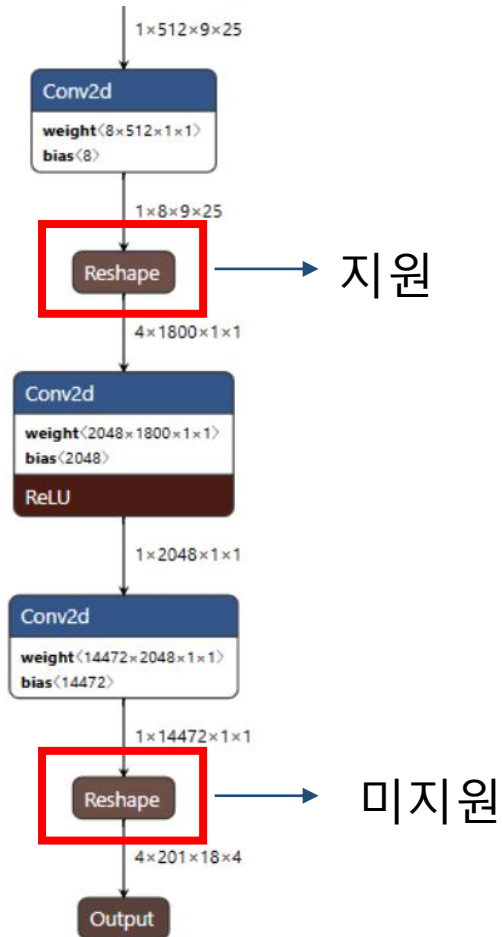
- 양자화 시 다음과 같이 reshape 레이어에서 에러 발생

UFLD 모델 변환

- 문제 원인 분석
 - Netron을 통해 UFLD 모델의 레이어 분석
 - \$ netron input/UFLD.onnx



UFLD 모델 변환

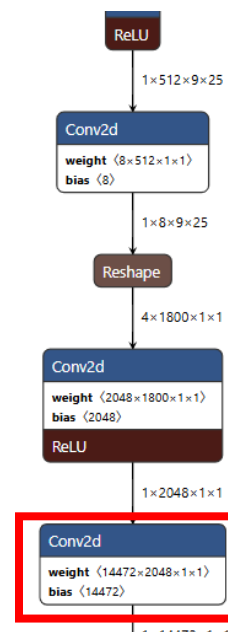


문제 원인 분석

- 모델 하단에 reshape layer 존재 확인
- Reshape의 경우 대표적으로 지원하지 않는 레이어
- Flatten의 기능에 해당하는 레이어의 경우 지원
- Flatten -> 2,3D 특징맵을 1D 벡터로 펼치는 연산
- 하지만 데이터의 순서를 재배열 하는 reshape는 지원하지 않음

UFLD 모델 변환

- Reshape 레이어 이름 확인
 - Netron을 통해 이전에 변환한 UFLD.enlight 파일 확인
 - `$ netron output_networks/UFLD.enlight`
 - 최종 출력으로 설정할 레이어 클릭 후 이름 확인
 - Reshape 레이어를 제거한 바로 위의 Conv2d 레이어
Linear_1__Gemm_51 레이어를 최종 출력 설정



NODE PROPERTIES	
type	Conv2d
name	Linear_1__Gemm_51
ATTRIBUTES	
in_channels	2048
out_channels	14472
kernel_size	1, 1
padding	0, 0
stride	1, 1
dilation	1, 1
groups	1
dataLayout	NCHW
qbit	8
INPUTS	
40	id: 40

UFLD 모델 변환

- Reshape 레이어 제거 & 모델 컨버팅 수행

```
$ python3 EnlightSDK/converter.py input/UFLD.onnx --type unknown --dataset Custom --dataset-root my_dataset_path --output output_networks/UFLD.enlight --enable-track --force-output Linear_1__Gemm_51
```

- 옵션 설명

- type : 후처리 타입 설정 (obj나 cls가 아니기에 unknown으로 설정)
- dataset : 양자화 스케일 값 추출을 위한 데이터셋
- output : 변환된 모델 output
- enable-track : 양자화 스케일 수집을 위한 옵션 활성화
- Force-output : 레이어 제거를 위해 최종 출력으로 사용할 레이어 설정

UFLD 모델 변환

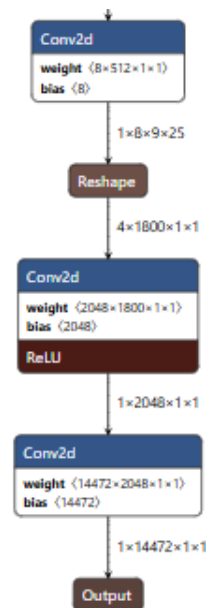
- 모델 양자화 수행

```
$ python3 EnlightSDK/quantizer.py output_networks/UFLD.enlight --output
output_networks/UFLD_quantized.enlight
```

```
Start BatchormalizeFold optimizer
optimizing ..... done.
Start FuseConstantEltwLayer optimizer
optimizing ..... done.
Start TreatIntermediateOuput optimizer
optimizing ..... done.
Start ModifyLayerQuantizationFriendly optimizer
optimizing ..... done.
quantization..... done.
End sanity check for custom quantization
Start MakeCompilerFriendly optimizer
optimizing ..... done.
Start MakeLayerAligned optimizer
optimizing ..... done.
Serializing Graph done.
Writing to File..... Done
Writing File path : output_networks/UFLD_quantized.enlight

Quantizer done.
```

- 정상적으로 변환됨을 확인



UFLD 모델 변환

- 모델 컴파일 수행

```
$ python3 EnlightSDK/compiler.py output_networks/UFLD_quantized.enlight
```

```
code-emitting ..... done.
work-buffer allocation sanity check ..... done.
Start MakeCode(version=NPUV20) code-emitting
code-emitting ..... done.

                                ./output_code/UFLD_quantized/npu_cmd.h created
                                ./output_code/UFLD_quantized/npu_cmd.bin created
                                ./output_code/UFLD_quantized/npu_cmd.txt created
Writing [44721664]/[44721664]
                                ./output_code/UFLD_quantized/quantized_network.bin created
                                ./output_code/UFLD_quantized/oact_entry_table.py created
                                ./output_code/UFLD_quantized/network.h created
                                ./output_code/UFLD_quantized/post_process.c created
                                ./output_code/UFLD_quantized/oe_network.bin created
                                ./output_code/UFLD_quantized/oe_network.h created

                                NPU compiler: done sucessfully.

Compiler. done.
```

UFLD 후처리 과정

- 네트워크 및 후처리 빌드 수행

- 모델 복사

- \$ cd build_network

- \$ cp ../output_code/UFLD_quantized/ ./ -ar

- 링크된 파일 제거

- \$ rm -rf network.h post_process.c

- UFLD 네트워크 및 후처리 파일 링크

- \$ ln -s UFLD_quantized/network.h

- \$ ln -s UFLD_quantized/post_process.c

- 후처리 빌드 수행

- \$ make

```
aarch64-none-linux-gnu-gcc -shared -Wl,-soname,net.so -o net.so post_process.o detector.o classifier.o yolo_detector.o ssd_detector.o custom_postproc.o enlight_network.o network.o  
rm -rf ./*.o network/*.o enlight/*.o /*.o
```

UFLD 후처리 과정

- 네트워크 및 후처리 빌드 수행
 - 후처리 파일 복사 및 AI-G 보드 전송 (wsl ubuntu)

```
$ cp net.so UFLD_quantized
```

```
$ scp -r UFLD_quantized root@192.168.0.100:/home/root/ → 비밀번호 root 입력
```

UFLD 후처리 과정

- 네트워크 및 후처리 빌드 수행
 - AI-G 보드 준비 및 추론 명령어 입력 (AI-G Board)
`$ tcnnapp -n UFLD_quantized -p /dev/video2`
 - Netron에서 현재 출력 텐서가 1 x 14472 x 1 x 1 이므로 추론로그가 다음과 같이 출력
 - UFLD 모델은 후처리 연산이 모델에 포함되어 있지 않아 추가적인 후처리와 자른 레이어에 대한 연산을 진행해야 함

UFLD 후처리 과정

- 후처리 과정

- Obj나 cls의 경우 모델 내부의 postprocess.c와 build_network dir 내부의 classifier, detector내부에서 후처리가 이루어지도록 구성되어 있음

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/build_network/detector$ ls  
detector.c  detector.h  ssd_detector.c  yolo_detector.c
```

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/build_network/classifier$ ls  
classifier.c
```

- 해당 파일 내부에서 detector의 경우 nms, df1, bbox 처리와 같은 연산을 진행

UFLD 후처리 과정

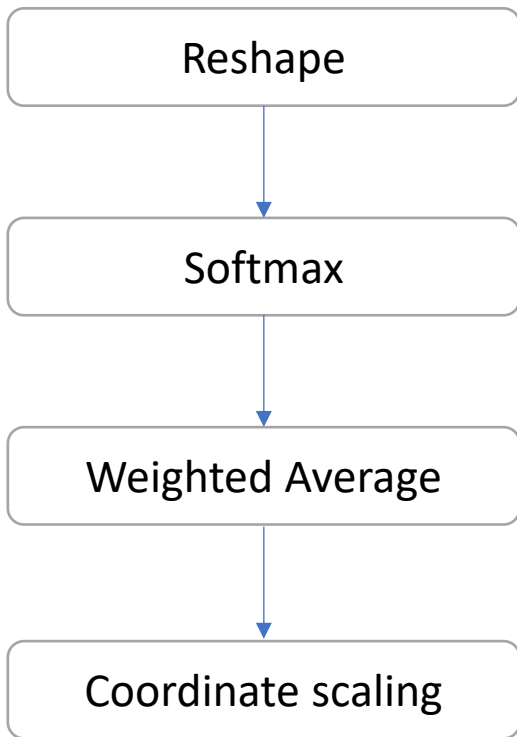
- 후처리 과정

- Custom 모델의 경우 사용자가 직접 후처리를 구성해야함
- Custom_postproc.c와 네트워크 파일 내부의 post_process.c 코드를 수정해서 후처리 진행

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/build_network/custom_postproc$ ls  
custom_postproc.c
```

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/build_network/UFLD_quantized$ ls  
compile_debug.log  network_group_info.csv  npu_cmd.bin  npu_cmd.txt  oe_network.bin  post_process.c  trap_point.py  
network_buf.txt    network.h               npu_cmd.h    oact_entry_table.py  oe_network.h    quantized_network.bin
```

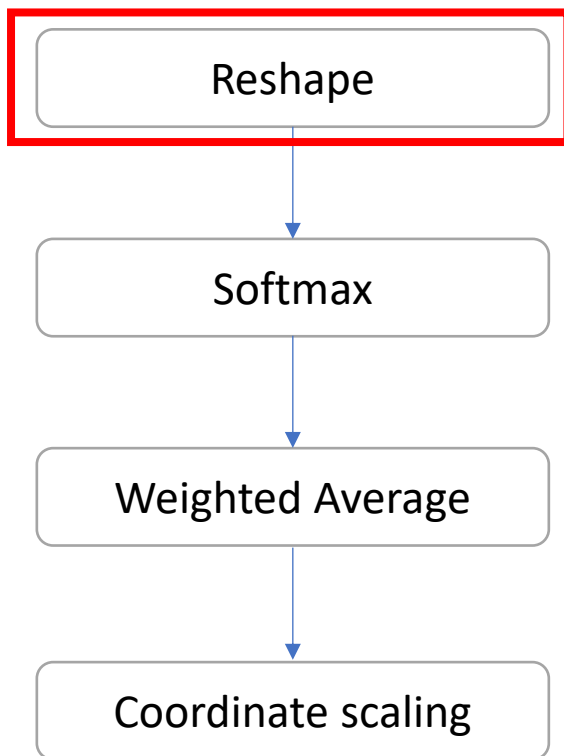
UFLD 후처리 과정



• UFLD 후처리 과정

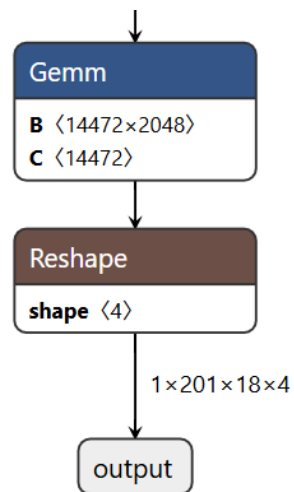
- Custom 모델의 경우 사용자가 직접 후처리를 구성해야함
- Custom_postproc.c와 네트워크 파일 내부의 post_process.c 코드를 수정해서 후처리 진행
- UFLD github의 demo.py라는 코드에서 출력 텐서에 대한 값을 좌표 값으로 변환하는 후처리 작업을 수행

UFLD 후처리 과정

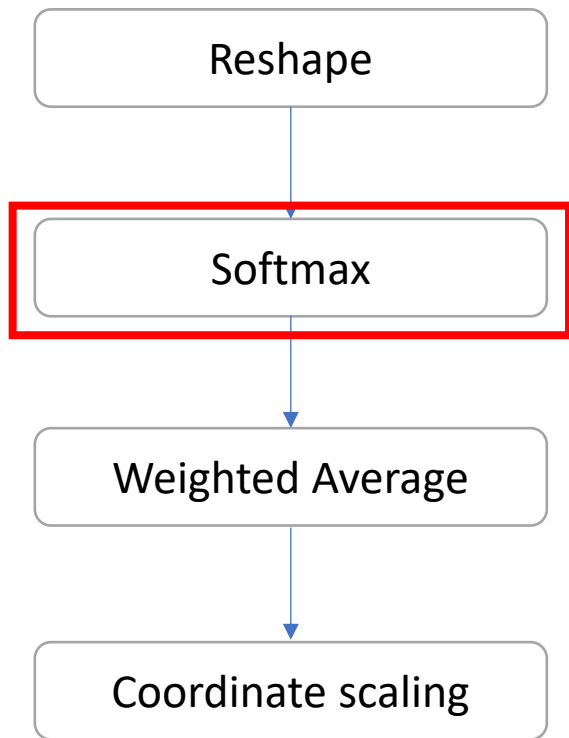


• Reshape

- 데이터를 $1 \times 14472 \times 1 \times 1$ 에서 $1 \times 201 \times 18 \times 4$ 로 재배열하는 연산 수행
- 해당 레이어를 제거하였기 때문에 reshape 연산을 코드로 작성



UFLD 후처리 과정



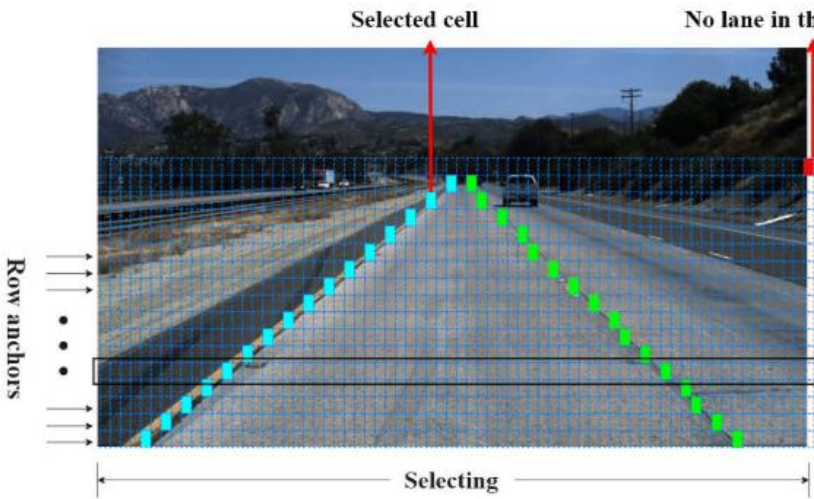
• Softmax

- 신경망의 출력 값을 확률로 바꾸는 연산
- 각 값에 e^x 를 취해 전체 합으로 나눠 합이 1이 되도록 함
- 즉, “가장 그럴듯한 위치가 어디일까?”를 확률로 표현
- UFLD 모델은 이미지를 여러 가로줄로 나누고 각 줄에서 차선이 있을 x 위치를 여러 칸으로 분류
- 즉, Softmax 연산을 거쳐 “줄마다 어느 칸에 차선이 있을 확”로 변환하게 됨

UFLD 후처리 과정

Reshape

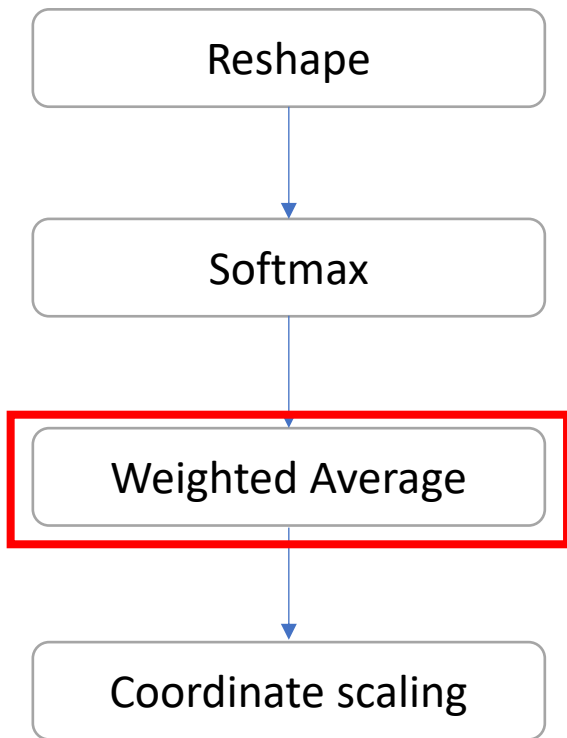
Softmax



• Softmax

- 신경망의 출력 값을 확률로 바꾸는 연산
- 각 값에 e^x 를 취해 전체 합으로 나눠 합이 1이 되도록 함
- 즉, “가장 그럴듯한 위치가 어디일까?”를 확률로 표현
- UFLD 모델은 이미지를 여러 가로줄로 나누고 각 줄에서 차선이 있을 x 위치를 여러 칸으로 분류
- 즉, Softmax 연산을 거쳐 “줄마다 어느 칸에 차선이 있을 확”로 변환하게 됨

UFLD 후처리 과정



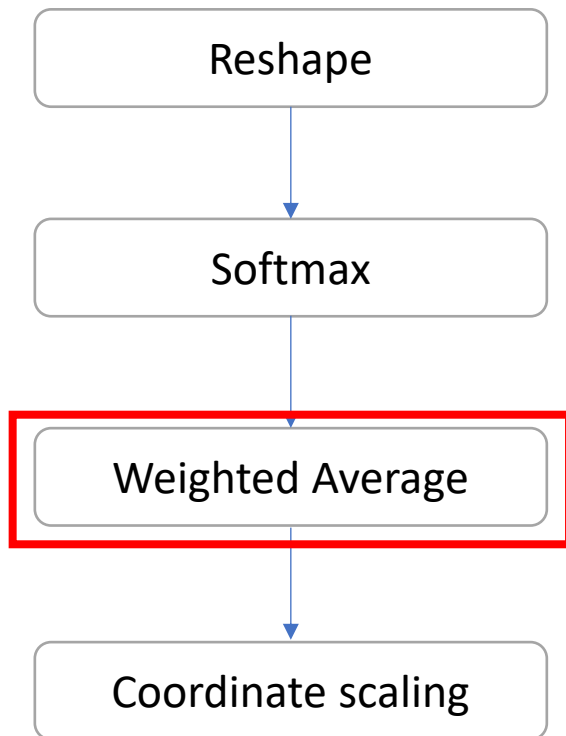
• Weighted Average

- Softmax로 각 칸의 확률을 얻은 뒤 Weighted Average로 x 좌표를 계산
- 확률이 높은 칸일수록 더 큰 비중을 두는 평균 연산

$$x_{\text{예측}} = \sum_{i=1}^w (\text{확률}_i \times \text{칸번호}_i)$$

- 왜 가장 큰 확률을 구하는 argmax를 쓰지 않을까?
- -> 가장 큰 확률 하나만 고르게 될 경우 좌표 값이 툭툭 끊기게 되는 현상 발생
- -> 가중 평균 합을 통해 좌표가 부드럽게 이동, 곡선이 자연스러움

UFLD 후처리 과정



• Weighted Average

- Softmax로 각 칸의 확률을 얻은 뒤 Weighted Average로 x 좌표를 계산
- 확률이 높은 칸일수록 더 큰 비중을 두는 평균 연산

$$x_{\text{예측}} = \sum_{i=1}^w (\text{확률}_i \times \text{칸번호}_i)$$

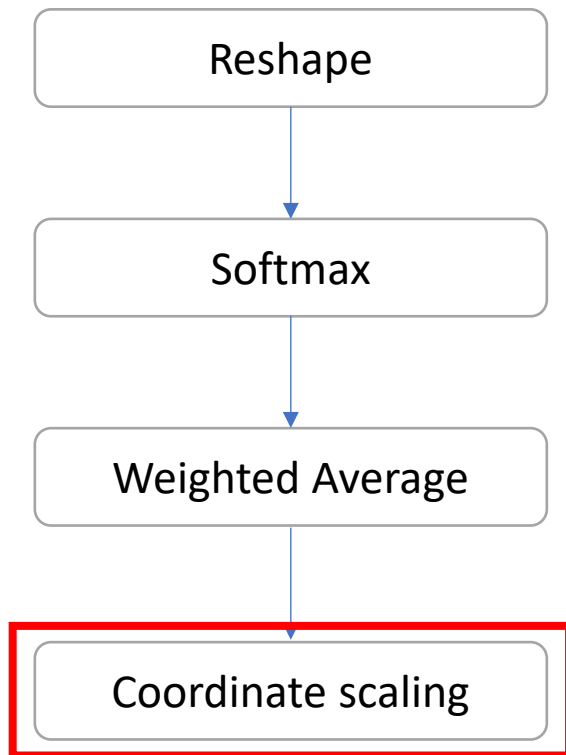
칸 번호 : [1, 2, 3]

Softmax 확률 : [0.2, 0.5, 0.3]

➔ Weighted Average = $(1 \times 0.2 + 2 \times 0.5 + 3 \times 0.3) = 2.1$

➔ 예측된 x 위치 = 2.1번째 칸

UFLD 후처리 과정



- Coordinate scaling

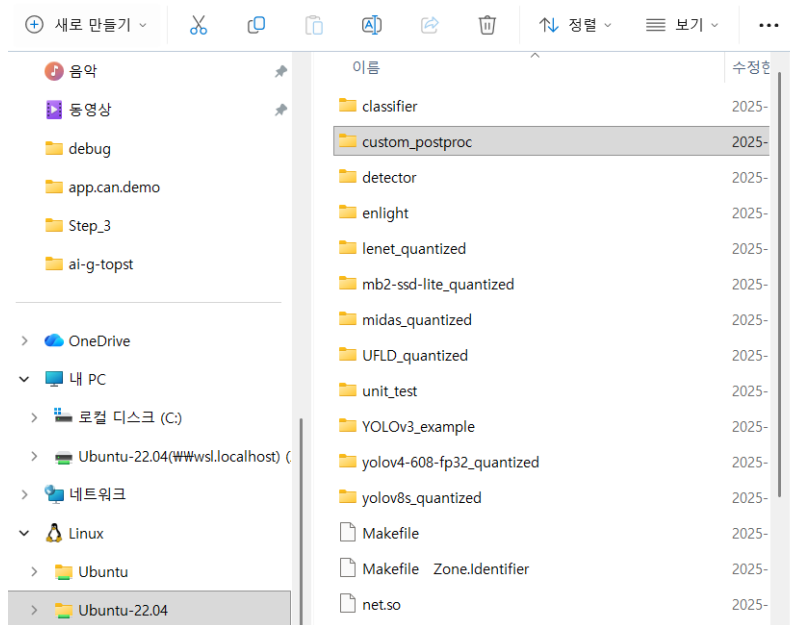
- 모델 입력이 288x800이기 때문에 계산된 좌표 값을 실제 이미지 크기에 맞게 변환

$$x_{pixel} = \frac{x_{grid}}{grid_num} \times image_width$$

- 해당 과정을 통해 예측된 차선을 화면위에 정확하게 표시

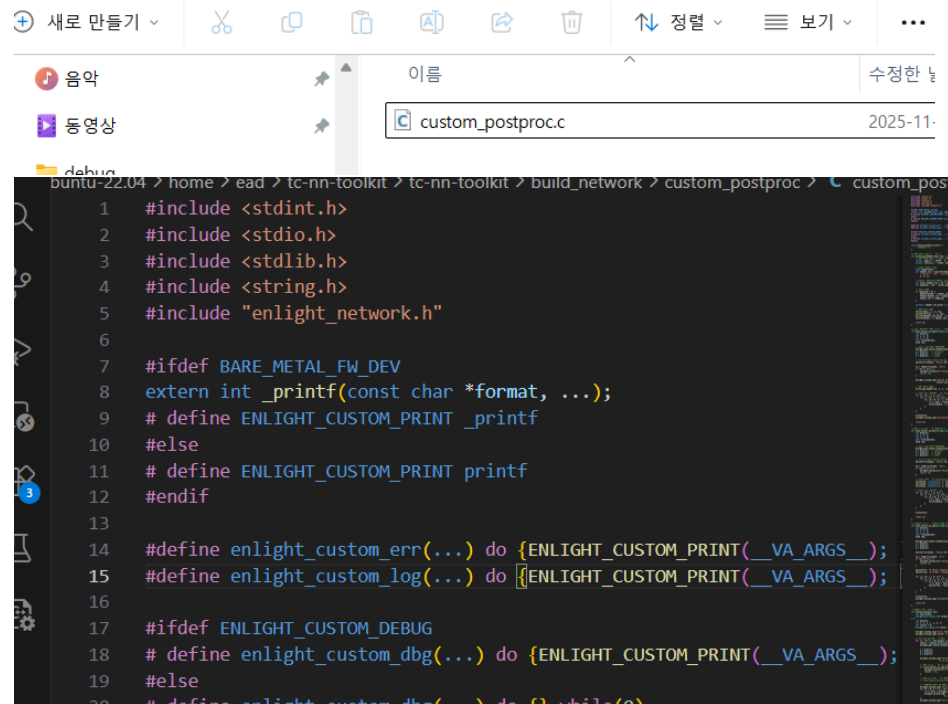
UFLD 후처리 과정

- 후처리 코드 작성
 - 파일 탐색기에 ubuntu22.04 탭 선택
 - Home -> {user} -> tc-nn-toolkit -> build_network -> custom_postproc 선택



UFLD 후처리 과정

- 후처리 코드 작성
 - custom_postproc.c 파일 열기 (vscode)



```
1 #include <stdint.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include "enlight_network.h"
6
7 #ifdef BARE_METAL_FW_DEV
8 extern int _printf(const char *format, ...);
9 # define ENLIGHT_CUSTOM_PRINT _printf
10 #else
11 # define ENLIGHT_CUSTOM_PRINT printf
12 #endif
13
14 #define enlight_custom_err(...) do {ENLIGHT_CUSTOM_PRINT(__VA_ARGS__);}
15 #define enlight_custom_log(...) do {ENLIGHT_CUSTOM_PRINT(__VA_ARGS__);}
16
17 #ifdef ENLIGHT_CUSTOM_DEBUG
18 # define enlight_custom_dbg(...) do {ENLIGHT_CUSTOM_PRINT(__VA_ARGS__);}
19 #else
20 # define enlight_custom_dbg(...) do {} while(0)
```

UFLD 후처리 과정

• custom_postproc.c 작성

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <float.h>
#include "enlight_network.h"

void custom_softmax(float* input, float* output, int
axis_len) {
    for (int j = 0; j < 18; j++) {
        for (int k = 0; k < 4; k++) {
            float max_val = input[0 * 72 + j * 4 + k];
            for (int i = 1; i < axis_len; i++) {
                if (input[i * 72 + j * 4 + k] > max_val)
                {
                    max_val = input[i * 72 + j * 4 + k];
                }
            }
            float sum = 0.0f;
            for (int i = 0; i < axis_len; i++) {
                output[i * 72 + j * 4 + k] = expf(input[i
* 72 + j * 4 + k] - max_val);
                sum += output[i * 72 + j * 4 + k];
            }
            for (int i = 0; i < axis_len; i++) {
                output[i * 72 + j * 4 + k] /= sum;
            }
        }
    }
}
```

```
void custom_postproc_init() {
    // Initialization code if needed
}

int custom_postproc_run(
    int num_output,
    enlight_act_tensor_t** output_tensors,
    float* loc
) {
    int dims[4];
    enlight_act_tensor_t* tensor = output_tensors[0];

    enlight_get_tensor_dimensions(tensor, dims);
    int batch_size = dims[0];
    int num_classes = dims[1];
    int h = dims[2];
    int w = dims[3];

    float* origin_data =
(float*)malloc(output_tensors[0]->size * sizeof(float));
    enlight_get_tensor_data(output_tensors[0],
origin_data);

    int griding_num = 200;
    float* out_j = (float*)malloc(14472 * sizeof(float));
    float* prob = (float*)malloc(14472 * sizeof(float));

    memcpy(out_j, origin_data, sizeof(float) * 14472);
}
```

UFLD 후처리 과정

• custom_postproc.c 작성

```
for (int i = 0; i < 201; i++) {
    for (int j = 0; j < 18; j++) {
        memcpy(&out_j[i * 72 + (17 - j) * 4],
            &origin_data[i * 72 + j * 4], sizeof(float) * 4);
    }

    custom_softmax(out_j, prob, 200);

    int* idx = (int*)malloc(griding_num * sizeof(int));
    for (int i = 0; i < griding_num; i++) {
        idx[i] = i + 1;
    }

    for (int j = 0; j < 18; j++) {
        for (int k = 0; k < 4; k++) {
            loc[j * 4 + k] = 0.0f;
            for (int i = 0; i < griding_num; i++) {
                loc[j * 4 + k] += prob[i * 72 + j * 4 + k]
* idx[i];
            }
        }
    }
}
```

```
float* out_j_argmax = (float*)malloc(18 * 4 *
sizeof(float));
for (int j = 0; j < 18; j++) {
    for (int k = 0; k < 4; k++) {
        float max_val = -FLT_MAX;
        int max_idx = -1;
        for (int i = 0; i < griding_num + 1; i++) {
            float value = out_j[i * 72 + j * 4 + k];
            if (value > max_val) {
                max_val = value;
                max_idx = i;
            }
        }
        out_j_argmax[j * 4 + k] = max_idx;
    }
}

for (int j = 0; j < 18; j++) {
    for (int k = 0; k < 4; k++) {
        if (out_j_argmax[j * 4 + k] == 200) {
            loc[j * 4 + k] = 0;
        }
    }
}

float x_scale = 800.0f / (griding_num - 1); //
Assuming input width is 800
float y_scale = 288.0f / 17; // Assuming input
height is 288 and 18 anchors
```

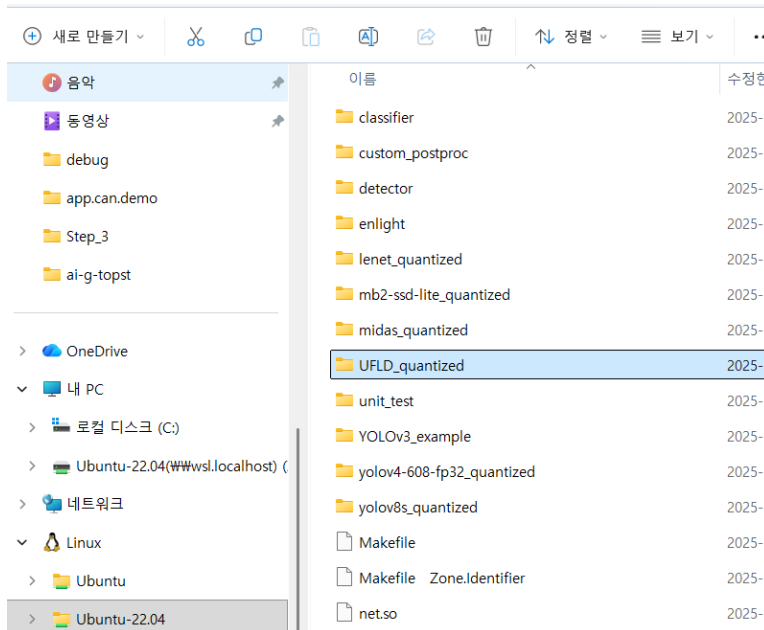
UFLD 후처리 과정

- custom_postproc.c 작성

```
for (int j = 0; j < 18; j++) {  
    for (int k = 0; k < 4; k++) {  
        if (loc[j * 4 + k] > 0) { // Valid point  
            float x = loc[j * 4 + k] * x_scale;  
            float y = j * y_scale;  
        }  
    }  
}  
  
free(origin_data);  
free(out_j);  
free(prob);  
free(idx);  
free(out_j_argmax);  
  
return 0;  
}
```

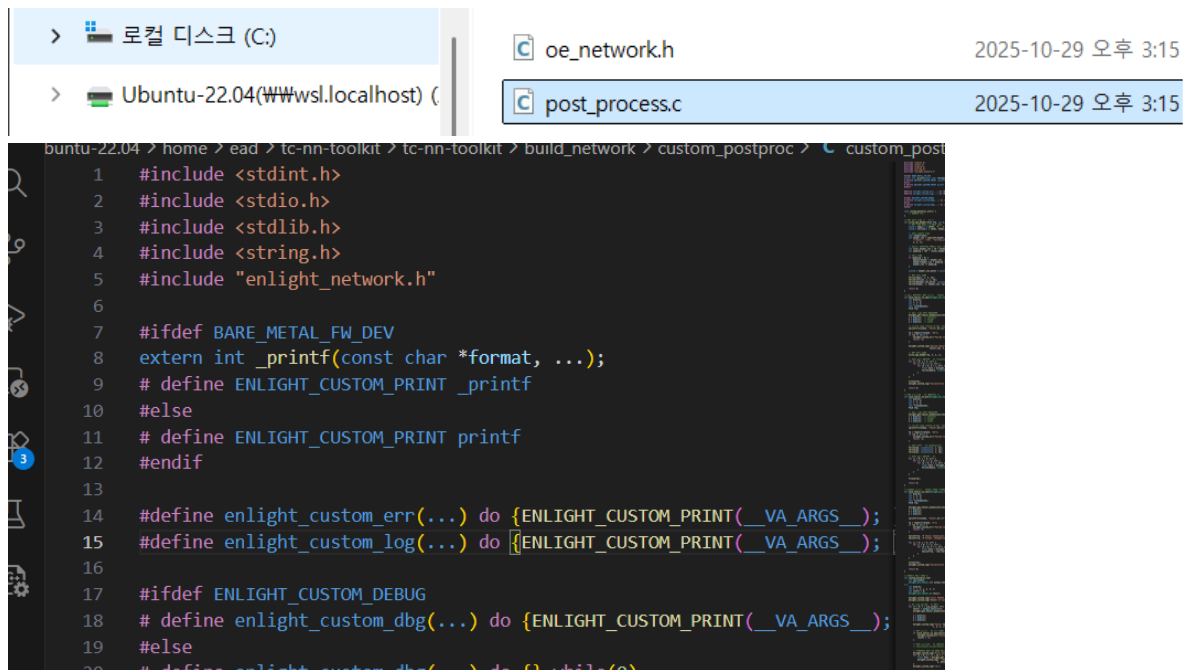
UFLD 후처리 과정

- 후처리 코드 작성
 - 파일 탐색기에 ubuntu22.04 탭 선택
 - Home -> {user} -> tc-nn-toolkit -> build_network -> UFLD_quantized선택



UFLD 후처리 과정

- 후처리 코드 작성
 - post_process.c파일 열기 (vscode)



The screenshot shows the Visual Studio Code interface. The file explorer on the left displays the project structure, with the file `post_process.c` selected under the `Ubuntu-22.04(wsl:localhost)` workspace. The main editor area shows the content of `post_process.c`, which includes standard C headers and custom print/log macros for the Enlightenment framework.

```
buntu-22.04 > home > ead > tc-nn-toolkit > tc-nn-toolkit > build_network > custom_postproc > custom_post  
1  #include <stdint.h>  
2  #include <stdio.h>  
3  #include <stdlib.h>  
4  #include <string.h>  
5  #include "enlight_network.h"  
6  
7  #ifdef BARE_METAL_FW_DEV  
8  extern int _printf(const char *format, ...);  
9  # define ENLIGHT_CUSTOM_PRINT _printf  
10 #else  
11 # define ENLIGHT_CUSTOM_PRINT printf  
12 #endif  
13  
14 #define enlight_custom_err(...) do {ENLIGHT_CUSTOM_PRINT( __VA_ARGS__ );  
15 #define enlight_custom_log(...) do {ENLIGHT_CUSTOM_PRINT( __VA_ARGS__ );  
16  
17 #ifdef ENLIGHT_CUSTOM_DEBUG  
18 # define enlight_custom_dbg(...) do {ENLIGHT_CUSTOM_PRINT( __VA_ARGS__ );  
19 #else  
20 # define enlight_custom_dbg(...) do { } while(0)
```

UFLD 후처리 과정

• post_process.c 작성

```
#include "stdint.h"
#include "enlight_network.h"
#include <stdlib.h>
#include <string.h>
#ifdef __i386__
#include <stdio.h>
#define post_process_log(...) do {printf(__VA_ARGS__);}
while(0)
#else
#define post_process_log(...) do {_printf(__VA_ARGS__);}
while(0)
#endif

extern void custom_postproc_init();

typedef struct {
    float loc[72];
}custom_lane_t;
```

```
int run_post_process(
    void *net_inst,
    void *output_base,
    int num_input,
    custom_lane_t *custom_loc
){
    enlight_network_t *inst;
    enlight_custom_postproc_t *custom_param;
    int result;
    int i,num_output;
    enlight_act_tensor_t
*output_tensors[MAX_NUM_OUTPUT];

    inst = (enlight_network_t *)net_inst;
    custom_param = (enlight_custom_postproc_t *)inst->
post_proc_extension;

    num_output
=enlight_custom_get_output_tensors(custom_param,
output_tensors);

    for (i = 0; i < num_output; i++) {
        output_tensors[i]->base =
output_base;
    }
```

UFLD 후처리 과정

- post_process.c 작성

```
custom_postproc_init();

float *loc = (float *)malloc(18 * 4 * sizeof(float));
if (!loc) {
    fprintf(stderr, "Failed to allocate memory for loc\n");
    return -1;
}

result = custom_postproc_run(num_output, output_tensors, loc);
memcpy(custom_loc->loc, loc, 72 * sizeof(float));
free(loc);

return num_output;
}
```

UFLD 후처리 과정

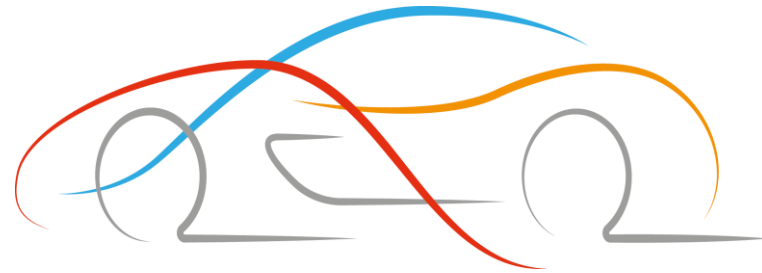
- 네트워크 및 후처리 빌드 수행 (ws1 우분투에서 진행)

```
$ make
```

- 후처리 파일 복사 및 AI-G 보드 전송

```
$ cp net.so UFLD_quantized
```

```
$ scp -r UFLD_quantized root@192.168.0.100:/home/root/ → 비밀번호 root 입력
```



Thank You